

(V2) API Observability Platform.

V2 API Observability Platform: The Beginner's Guide

Welcome to the codebase! This platform acts like a highly intelligent security guard for your company's APIs. It watches the traffic (like how fast the API responds), learns what "normal" looks like, and raises an alarm if things get weird (like a sudden spike in errors or a change in the data format).

1. Keywords & Concepts (The Jargon Buster)

Before reading the code, let's define some terms you will see everywhere:

- * **API (Application Programming Interface):** A waiter in a restaurant. The customer (user) gives an order, the waiter takes it to the kitchen (server), and brings back the food (data).
- * **Latency / P95 Latency:** Latency is how long the API takes to respond. "P95" means the 95th percentile. If P95 latency is 200ms, it means 95% of all requests were faster than 200ms, and 5% were slower. We track P95 to ignore tiny, random hiccups and focus on real slowdowns.
- * **Time-Series Data:** Data recorded over time. Think of a stock market chart. Our data is "API Latency at 1:00 PM, 1:15 PM, 1:30 PM, etc."
- * **DataFrame:** A digital spreadsheet inside Python. It has rows and columns. We use a library called **Polars** to make these spreadsheets, which is much faster than the older standard (Pandas).
- * **Ensemble AI / Consensus:** Instead of relying on one AI model to make a decision, we use three. If 2 out of 3 agree that something is an anomaly, we sound the alarm. This prevents "false alarms."
- * **Data Drift:** When the actual numbers in your data slowly change over time. For example, if your average user's account balance is usually \$1,000, but suddenly it jumps to \$5,000, the data has "drifted."
- * **JSON Schema:** The "blueprint" of your data. If your data usually has `{"name": "John", "age": 30}`, and suddenly a request comes in as `{"name": "John", "location": "NY"}`, the schema (blueprint) has changed.

🗨️ 2. The Time-Series Models Explained

In V2, we use three different mathematical "brains" to look at our data. Here is how they work in plain English:

Model 1: Isolation Forest (Library: PyOD)

* **How it works:** Imagine a forest with thousands of normal pine trees, and one weird, crooked oak tree. If you draw random lines through the forest, it is very easy to "isolate" the weird tree because it stands out.

* **In our code:** It looks at a scatterplot of our API latency and error rates. If a data point is far away from the rest of the cluster, it flags it as an anomaly. It is a traditional, statistical machine learning model.

Model 2: Amazon Chronos (Hugging Face TSFM)

* **How it works:** Think of ChatGPT. ChatGPT reads a sentence ("The sky is...") and guesses the next word ("blue"). Amazon Chronos does the exact same thing, but with numbers! It is a **Time-Series Foundation Model (TSFM)**.

* **In our code:** We give it the last 20 latency numbers. It uses a massive neural network to guess the next 4 numbers. It gives us a "confidence band" (e.g., "I am 90% sure the next latency will be between 100ms and 150ms"). If the *actual* latency turns out to be 800ms, Chronos screams "ANOMALY!" This is called **Zero-Shot Forecasting**.

Model 3: IBM TinyTimeMixer / TTM (Hugging Face TSFM)

* **How it works:** Similar to Chronos, but built by IBM to be incredibly lightweight and fast.

* **In our code:** For this specific POC (Proof of Concept), to ensure it runs smoothly on a standard desktop without crashing, we wrote a "statistical fallback" for TTM. It calculates a rolling average and flags anything that jumps too high (2.5 standard deviations above the average).

📄 3. Understanding the Mock Data Files

Because we don't have a real production database connected right now, we generated "Mock" (fake) data to test our code.

A. `mock\_es\_response.json` (The API Traffic)

This simulates what Elasticsearch (our database) returns when we ask: **"Give me the API traffic for the last 24 hours, grouped into 15-minute buckets."**

****Sample Record:****

```
```json
{
 "key_as_string": "2026-07-10T19:42:51.060408Z",
 "doc_count": 3291,
 "p95_latency": { "values": { "95.0": 286.84 } },
 "error_count": { "value": 38 }
}
```
```

*** **Meaning:**** On July 10th at 19:42, there were ****3,291**** API requests (`doc\_count`). 95% of them finished in ****286.84 milliseconds**** or less. However, ****38**** of those requests failed with an error.

B. `payloads\_today.json` (The User Data)

This simulates the actual data payloads sent by users to the API.

****Sample Record:****

```
```json
{
 "transaction_id": "txn_0",
 "user": {
 "account_type": "enterprise",
 "account_balance": 5241.24,
 "kyc_verified": true
 },
 "metadata": {}
}
```
```

* **Meaning:** A user made a transaction. We track their balance. **Scenario:** In `payloads_yesterday.json`, the balances were around \$1,000. Today, they are around \$5,000. Our code will catch this "Data Drift". Also, notice `kyc_verified` is here? It wasn't here yesterday. Our code will catch this "Schema Change".

C. `v2_ensemble_results.json` (The Final Output)

This is the file our Python engine creates after doing all the math. It is what the UI reads.

Sample Record:

```
```json
{
 "timestamp": "2026-07-11 00:57:51",
 "p95_latency": 949.94,
 "anomaly_iforest": true,
 "anomaly_chronos": true,
 "anomaly_ttm": true,
 "consensus_anomaly": true
}
```
```

* **Meaning:** At 00:57, latency spiked to 949ms! Isolation Forest saw it (`true`), Chronos saw it (`true`), and TTM saw it (`true`). Because at least 2 out of 3 agreed, the final `consensus_anomaly` is `true`.

 4. Detailed Code Explanation (File by File)

Let's walk through the actual Python code.

 `config/config.yaml`

What it is: The control panel. Instead of hardcoding numbers into Python, we put them here. If you want to turn off a model, you just change `enabled: true` to `false` here, and the Python code automatically adapts.

- * ``metrics``: Tells the code to look at 15-minute intervals.
- * ``anomaly_detection``: Lists our 3 models (Isolation Forest, Chronos, TTM).
- * ``payload_monitoring``: Tells the code to use ``deepdiff`` (to find schema changes) and ``evidently`` (to find data drift in the ``user.account_balance`` field).

📦 ``requirements.txt``

****What it is:**** The grocery list for Python. It tells your computer exactly which external libraries to download to make the code work (like ``polars``, ``torch`` for neural networks, and ``streamlit`` for the UI).

💬 ``src/backend/processor.py`` (The Data Cleaner)

****Purpose:**** Takes the messy, nested JSON from Elasticsearch and turns it into a clean, flat Polars DataFrame (spreadsheet).

```
```python
```

```
import polars as pl
```

```
import logging
```

```
class DataProcessor:
```

```
 def process_es_aggregations(self, es_response: dict) -> pl.DataFrame:
```

```
 # 1. Dig into the JSON to find the list of time buckets
```

```
 buckets = es_response["aggregations"]["api_traffic_over_time"]["buckets"]
```

```
 flattened_data = []
```

```
 # 2. Loop through every 15-minute bucket
```

```
 for bucket in buckets:
```

```
 timestamp = bucket.get("key_as_string")
```

```
 request_count = bucket.get("doc_count", 0)
```

```
 # 3. Safely extract nested data. If it's missing, default to 0.0
```

```

p95_latency = None
if "p95_latency" in bucket and "values" in bucket["p95_latency"]:
 p95_latency = bucket["p95_latency"]["values"].get("95.0")

error_count = bucket.get("error_count", {}).get("value", 0)

4. Calculate the error rate (Errors divided by Total Requests)
error_rate = (error_count / request_count) if request_count > 0 else 0.0

5. Add this clean row to our list
flattened_data.append({
 "timestamp": timestamp,
 "request_count": request_count,
 "p95_latency": p95_latency if p95_latency is not None else 0.0,
 "error_rate": error_rate
})

6. Convert the list into a super-fast Polars DataFrame
df = pl.DataFrame(flattened_data)

7. Tell Polars that the "timestamp" column is an actual Date/Time, not just text
df = df.with_columns(
 pl.col("timestamp").str.strptime(pl.Datetime, "%Y-%m-%dT%H:%M:%S%.fZ", strict=False)
)
return df
'''

'''

🌲 `src/backend/ml_anomaly.py` (Model 1: Isolation Forest)
Purpose: Finds statistical outliers in the data.

```

```

```python
import polars as pl
from pyod.models.ifoorest import IForest

class AnomalyDetector:
    def __init__(self, contamination: float = 0.05):
        # Contamination = 0.05 means we expect roughly 5% of our data to be anomalies
        self.model = IForest(contamination=contamination, random_state=42)

    def detect_anomalies(self, df: pl.DataFrame, features: list[str]) -> pl.DataFrame:
        # 1. AI models can't read Polars DataFrames directly.
        # We extract just the numbers (latency, error_rate) and turn them into a NumPy array (a math
        matrix).
        X = df.select(features).fill_null(0.0).to_numpy()

        # 2. .fit(X) tells the model to study the data and learn what is "normal"
        self.model.fit(X)

        # 3. .predict(X) asks the model to label every row as 0 (Normal) or 1 (Anomaly)
        predictions = self.model.predict(X)

        # 4. Convert the 0s and 1s into True/False, and attach it as a new column to our spreadsheet
        is_anomaly_series = pl.Series("is_anomaly", predictions == 1)
        return df.with_columns([is_anomaly_series])
...

---

### 🤖 `src/backend/tsfm_inference.py` (Models 2 & 3: Hugging Face AI)
**Purpose:** Loads massive neural networks to predict the future and catch anomalies.

```

```

```python
import torch

from chronos import ChronosPipeline

class TSFMEsemble:

 def __init__(self, config: dict):

 # 1. Load the Amazon Chronos model from our local hard drive into memory using PyTorch
 self.pipelines["chronos"] = ChronosPipeline.from_pretrained(
 "amazon/chronos-t5-small",
 device_map="cpu", # Run on the standard computer processor
 torch_dtype=torch.float32
)

 def _run_chronos_inference(self, df: pl.DataFrame, data: np.ndarray, target_col: str) ->
 pl.DataFrame:

 # 2. We use a "Sliding Window". We look at 20 past data points to predict the next 4.
 context_len = 20
 step = 4
 anomalies = np.zeros(len(data), dtype=bool) # Create a blank list of Falses

 for i in range(0, len(data) - context_len - step, step):

 # Grab the past 20 points (Context) and the actual next 4 points (Future)
 context_data = data[i : i + context_len]
 actual_future = data[i + context_len : i + context_len + step]

 # Convert to a PyTorch Tensor (the format neural networks understand)
 context_tensor = torch.tensor(context_data, dtype=torch.float32).unsqueeze(0)

 # 3. Ask Chronos to predict the future 20 different times (num_samples=20)
 forecast = pipeline.predict(context_tensor, prediction_length=step, num_samples=20)

```



```

4. Find the 10th and 90th percentiles of those guesses to create a "Confidence Band"
forecast_numpy = forecast[0].numpy()
low_band, high_band = np.quantile(forecast_numpy, [0.1, 0.9], axis=0)

5. Check: Did the ACTUAL latency fall outside the predicted band? If yes, it's an anomaly!
for j in range(step):
 if actual_future[j] > high_band[j] or actual_future[j] < low_band[j]:
 anomalies[i + context_len + j] = True

Attach the results to the spreadsheet
return df.with_columns(pl.Series("anomaly_chronos", anomalies))
'''

```

### 🕵️ `src/backend/payload\_monitor.py` (The Schema & Drift Detective)

**\*\*Purpose:\*\*** Looks at the actual JSON data users send, not just how fast the API is.

```

```python
from deepdiff import DeepDiff
from evidently.report import Report
from evidently.metrics import ColumnDriftMetric

class PayloadMonitor:
    def detect_schema_changes(self, yesterday_data, today_data):
        # 1. DeepDiff compares the structure of yesterday's JSON to today's JSON.
        # ignore_order=True means if keys just swapped places, it won't trigger a false alarm.
        diff = DeepDiff(yesterday_schema, today_schema, ignore_order=True)
```

```
# It returns a dictionary showing exactly what fields were added or removed (e.g., 'kyc_verified'
added)
```

```
return {"fields_added": [...], "fields_removed": [...]}
```

```
def detect_data_drift(self, yesterday_data, today_data, target_fields):
```

```
    # 2. Evidently AI calculates the Population Stability Index (PSI).
```

```
    # It looks at the distribution of numbers (like account balances).
```

```
    # If the average jumps from $1000 to $5000, PSI goes up, and it flags "Data Drift Detected!"
```

```
    report = Report(metrics=[ColumnDriftMetric(column_name="user.account_balance",
stattest="psi")])
```

```
    report.run(reference_data=yesterday, current_data=today)
```

```
    return report.as_dict()
```

```
...
```

```
---
```

```
### 🎵 `src/backend/main_engine.py` (The Orchestrator)
```

```
**Purpose:** The boss. It reads the config, runs the data through all the models, and decides the
final consensus.
```

```
```python
```

```
class Orchestrator:
```

```
 def run_pipeline(self):
```

```
 # 1. Get Data
```

```
 df = processor.process_es_aggregations(es_response)
```

```
 # 2. Run Model 1 (Isolation Forest)
```

```
 df = detector.detect_anomalies(df)
```

```
 # 3. Run Models 2 & 3 (Chronos & TTM)
```

```
 df = tsfm_ensemble.detect_anomalies(df)
```

#### # 4. THE CONSENSUS LOGIC

# We add up the True/False values (True = 1, False = 0).

# If the sum is >= 2, it means at least 2 models agreed it was an anomaly!

```
consensus_series = (
 df["anomaly_iforest"].cast(pl.Int8) +
 df["anomaly_chronos"].cast(pl.Int8) +
 df["anomaly_ttm"].cast(pl.Int8)
) >= 2
```

# Save the final verdict to a file for the UI to read

```
df = df.with_columns(consensus_series.alias("consensus_anomaly"))
df.write_json("v2_ensemble_results.json")
```

...

---

###  `src/frontend/app.py` (The Streamlit Dashboard)

**\*\*Purpose:\*\*** The user interface. It reads the final JSON files and draws pretty charts.

```
```python
```

```
import streamlit as st
```

```
import plotly.express as px
```

1. Load the data the backend engine generated

```
df_pd = pl.read_json("v2_ensemble_results.json").to_pandas()
```

2. Draw a line chart of the Latency over time

```
fig = px.line(df_pd, x="timestamp", y="p95_latency")
```

3. If a model flagged an anomaly, draw a shape on that exact timestamp

Isolation Forest gets Blue Circles

```

anomalies_if = df_pd[df_pd["anomaly_iforest"] == True]

fig.add_trace(go.Scatter(x=anomalies_if["timestamp"], y=anomalies_if["p95_latency"],
mode="markers", marker=dict(color="blue"))))

# The True Consensus gets a massive Red X
consensus = df_pd[df_pd["consensus_anomaly"] == True]

fig.add_trace(go.Scatter(x=consensus["timestamp"], y=consensus["p95_latency"], mode="markers",
marker=dict(color="red", symbol="x", size=14)))

# Show the chart on the webpage
st.plotly_chart(fig)
'''

'''

```

🌟 Summary

1. **Data comes in** from the database (simulated by ``mock_es_data.py``).
2. **Polars** flattens it into a spreadsheet (``processor.py``).
3. **Three different AI brains** look at the spreadsheet and guess where the anomalies are (``ml_anomaly.py`` and ``tsfm_inference.py``).
4. **The Orchestrator** counts the votes. If 2 out of 3 brains agree, it's a confirmed anomaly (``main_engine.py``).
5. **Streamlit** draws a beautiful chart showing you exactly where the anomalies happened (``app.py``).